

ECE8803-AI4

Module: Fault Detection and Classification

Lecture 1: Representing a Process Run: PCA and Eigentraces

Course website: <https://aikhan123.github.io/khan-lab-website/ai-semiconductors-course.html>

Asif Khan

Associate Professor

School of Electrical and Computer Engineering

School of Materials Science and Engineering

Georgia Institute of Technology

akhan40@gatech.edu

What fault detection and classification is for

Every process run leaves a sensor record. FDC turns that record into a verdict on the run, before the wafers move on.

- 1 Equipment**
sensors sampled at 1–10 Hz for every recipe step
- 2 Historian**
traces stored with lot, wafer, chamber and recipe context
- 3 FDC engine**
scores each completed run against a model of normal
- 4 OCAP**
an out-of-control action plan routes the alarm to an engineer

Why the trace is used at all

Coverage. Metrology measures a few wafers per lot, days later. The trace exists for every run.

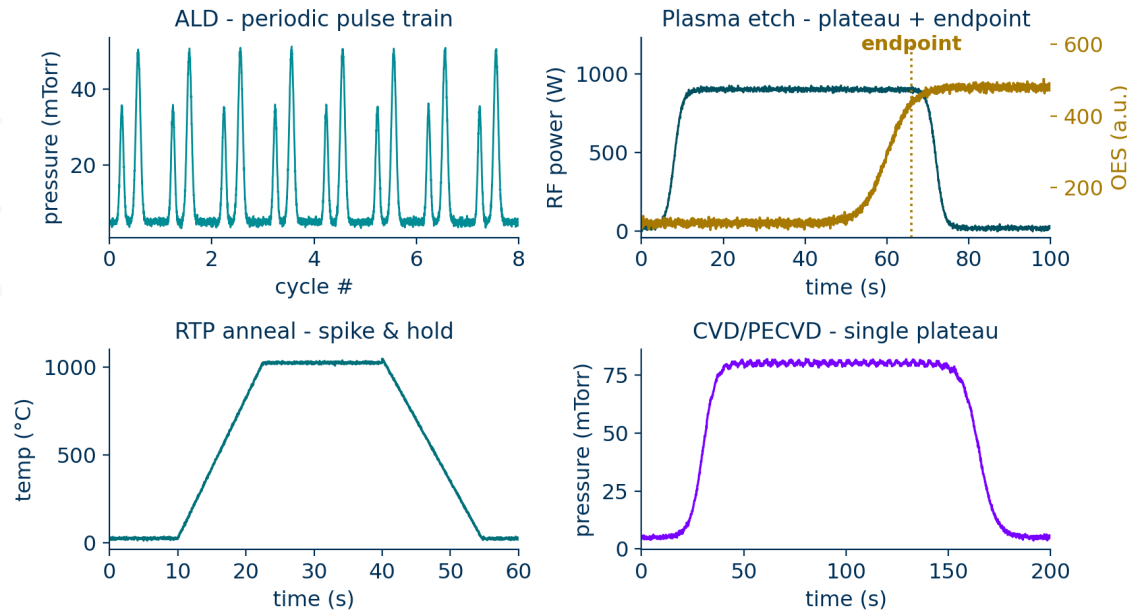
Timing. A run is scored between wafers, so a drifting chamber is caught within the shift.

Cost. The sensors are already installed and already logging.

Detection answers whether a run is abnormal. Classification answers what kind of abnormal it is. Lecture 2 covers detection.

Process traces across the fab

Four unit processes, four characteristic shapes. Every one of them produces a time series for a single recipe step.



Common structure

Bounded duration. A recipe step starts and ends at known points.

Repeated shape. The same step runs thousands of times a week.

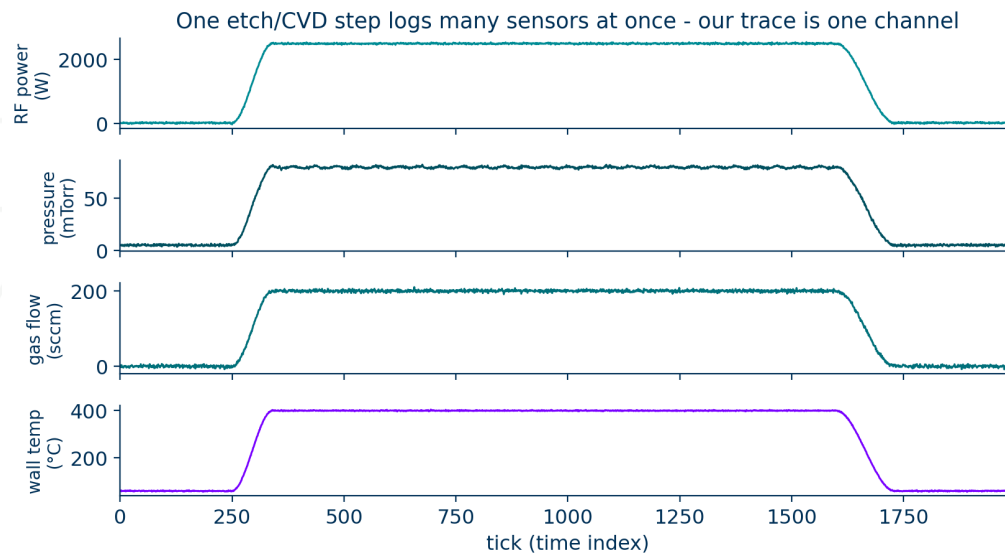
Controlled setpoints. A control loop drives the signal to a target and holds it.

Run-to-run variation. Timing, gain and noise differ between runs on a healthy tool.

Traces here are simulated with published recipe magnitudes. They stand in for logged data from a named tool.

One recipe step logs many channels

A single etch or CVD step records pressure, RF power, gas flow and temperature together.



This lecture models one channel

The channels move together. All of them follow the recipe envelope, so they are strongly correlated.

Faults can live in the relationship. Pressure and flow can each sit in range while their ratio is wrong.

One channel is the easier case. Everything shown here gets harder as channels are added.

100s

steps per wafer

1–10 Hz

per channel

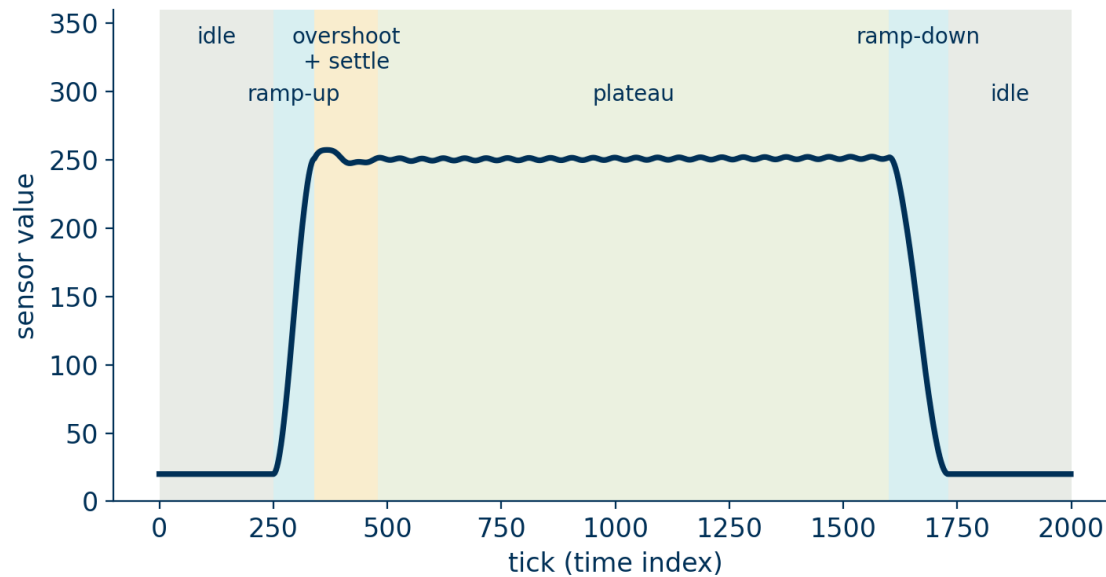
~1%

runs faulty

Scoring runs inline, between wafers, which puts a hard limit on how much computation a detector may use.

The trace used for the rest of the lecture

A single-plateau step, the CVD/PECVD shape from the previous slide, sampled at 2000 ticks.

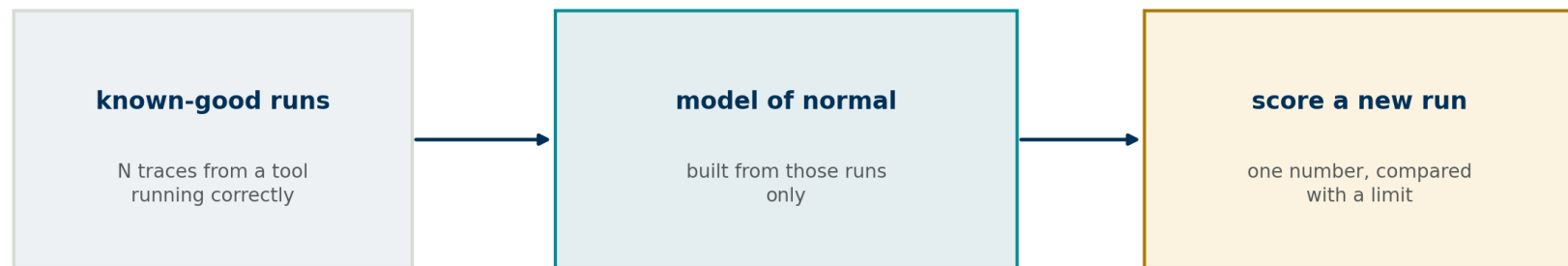


- 1 Idle**
steady at baseline
- 2 Ramp-up**
controlled rise to setpoint
- 3 Overshoot and settle**
the loop corrects, then damps
- 4 Plateau**
hold, with ripple and slow drift
- 5 Ramp-down**
return toward baseline

This deterministic shape is called the golden trace. Every detector in this lecture compares a run against it. Overshoot, ripple and drift are normal control-loop behaviour on a healthy tool.

Problem statement

Given traces from runs known to be good, decide whether a new run is normal.



No faulty run is available at any point in this diagram.

What you are given

N traces from healthy runs.

One channel, one recipe step, fixed length.

No faulty examples, and no fault labels.

Why the labels are missing

Faults are rare. About 1% of runs.

Faults are found late. Metrology results arrive days or weeks after the run.

The catalogue is open-ended. New failure modes appear as the tool ages.

A fault can hold a perfect shape

A mis-calibrated setpoint holds the plateau at the wrong level for the whole step.



10.4 units

injected offset, 4.5% of the 230-unit span

2.4 units

healthy plateau spread, 1σ across 100 runs

$\sim 4\times$

the offset, measured in units of that spread

every wafer

in the run sits out of spec

The ramps, the overshoot and the idle sections stay within normal variation. Only the plateau level moves.

The data used in these two lectures

All traces are generated from one deterministic shape with run-to-run variation added. Seeds are fixed, so every number in the deck reproduces.

the generative model — one function, five knobs

```
def render(t, p, rng):
    clean = golden(t + p['shift'],          # starts early or late
                  overshoot=p['os'], ripple=p['amp'],
                  phase=p['phi'], drift=p['drift'])
    y = BASE + p['gain'] * (clean - BASE) + p['offset']
    y += ou_wander(len(t), rng)             # slow sensor wander
    y += rng.normal(0, NOISE_STD, len(t))
    return y
```

- **Time shift** sigma 6 ticks, clipped at ± 15
- **Gain and offset** about 1% scaling plus a small bias
- **Measurement noise** white noise plus slow correlated wander
- **Ripple and drift** plateau ripple phase and drift vary per run

Two sets of runs

100 healthy runs. Generated with seed 20260723 and passed through a health gate that rejects any run whose idle, plateau or overshoot strays. Every measurement in Lecture 2 comes from these 100.

188 labelled runs. Generated with seed 20260730 from the same model, with one defect injected into 48 of them. These supply the faulty traces shown on the next few slides, and the scoring set for Lecture 3.

The faulty run on the previous slide came from the labelled set.

The dataset, laid out

One hundred runs. Each one is a trace of 2000 samples, produced by drawing a value for every knob and rendering.

run	trace	shift (ticks)	gain	offset (units)	ripple amp (units)	drift (units)
0		-0.38	0.9938	-1.40	1.02	0.95
1		+13.35	0.9922	-1.20	0.82	1.36
2		-0.74	0.9970	-0.40	1.16	1.42
⋮	⋮	⋮	⋮	⋮	⋮	⋮
98		-7.00	1.0174	+0.48	0.94	2.25
99		+0.14	0.9990	-0.17	0.84	2.00

X is a 100 x 2000 array of floats. One row per run, one column per tick. The knob columns exist because we generated the data.

What the model sees

The trace column only. A detector receives 2000 numbers per run and nothing else. The knob columns are available here because the data was generated, and they are used later to check what PCA found.

100
runs

2000
samples per run

200,000
numbers in total

A production baseline is built the same way, from runs an engineer has confirmed were good.

One hundred healthy runs

The same recipe step, repeated one hundred times. Ten of them are shown here, each plotted against the golden trace.

The first 10 of the 100 runs (faint gray = golden reference)



What repeats and what moves

The shape repeats every run.

Ramp timing shifts slightly from run to run.

Plateau level and ripple move a little.

All one hundred passed the health gate, so none of this is a defect.

100 runs

2000 ticks per run

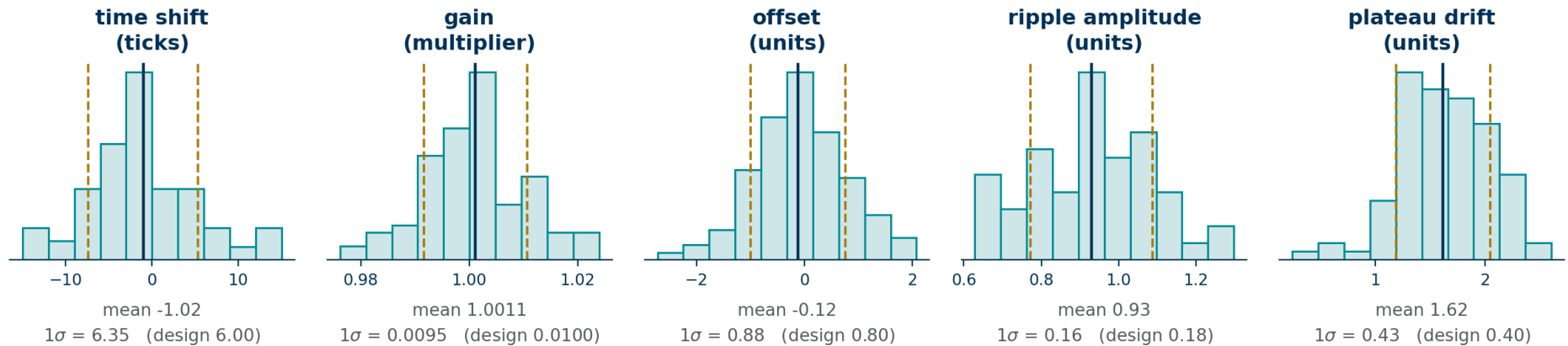
seed 20260723

every run passed the health gate

One hundred runs is a realistic baseline size. Lecture 3 measures what happens when the baseline is much smaller.

How the five knobs varied across the runs

Each histogram holds one hundred values. The solid line is the mean and the dashed lines sit one standard deviation either side.



Reading the spreads

The sample matches the design. With one hundred draws each measured standard deviation lands close to the value it was drawn from, and the sample means sit near zero.

The time shift is clipped at ±15 ticks, so the observed start times span 280 to 310.

Design values

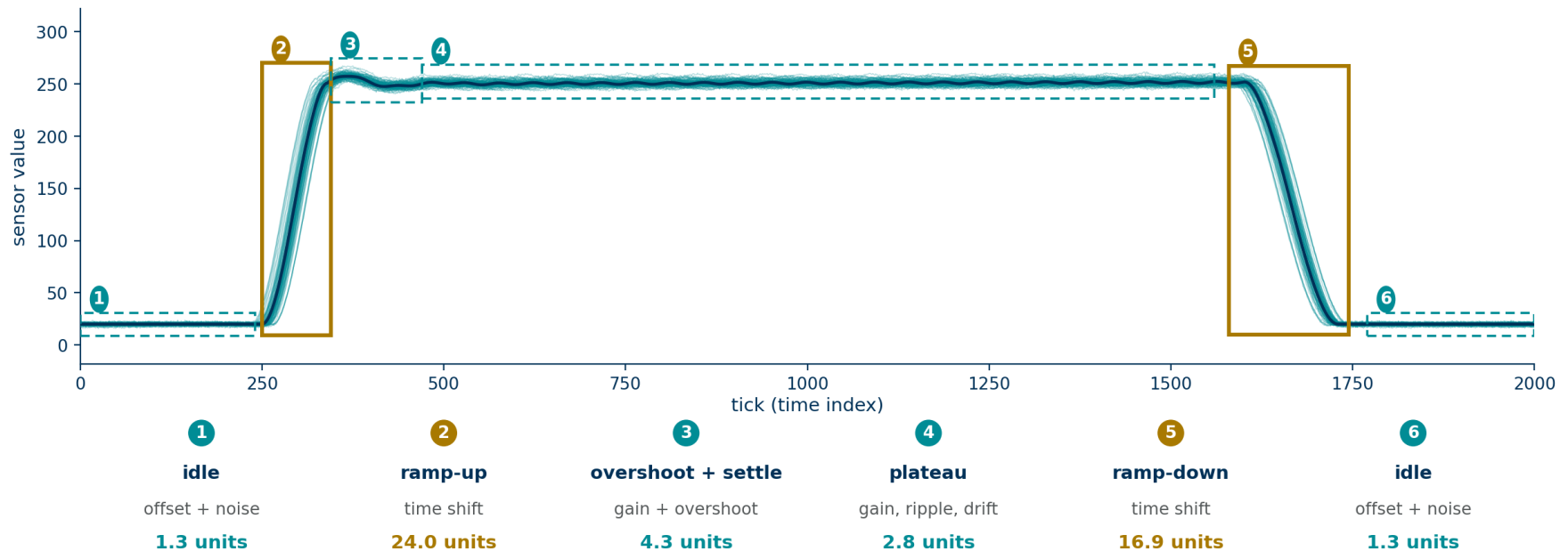
shift 6.0 ticks · gain 0.010 · offset 0.80 units

ripple amplitude 0.18 units · drift 0.40 units

Measurement noise is added per sample rather than per run: 0.55 units of white noise plus 0.35 units of slow wander.

Where the run-to-run variation sits

All one hundred runs overlaid. Each phase of the step is boxed, with the knob that dominates its spread and the size of that spread in sensor units.



The two ramps carry ten times the spread of every other phase, and all of it comes from the time shift. Sigma of 6 ticks is 0.3% of the run, and on a ramp climbing 3.9 units per tick that becomes 24 units. The setpoint fault from slide 7 moves the plateau by 10.

Six fault types, with their physical causes

Each red bundle is eight runs carrying one injected defect. The grey band behind them is the same hundred healthy runs.

— the same 100 healthy runs — faulty runs — 8 of each type — golden

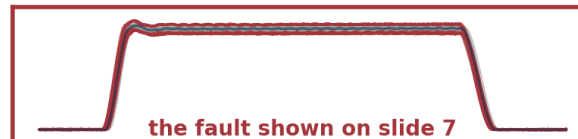
Slow ramp

a sticky valve, a slow mass-flow controller
lives on the rising edge



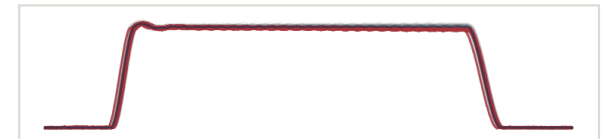
Setpoint offset

a mis-calibrated sensor or a drifting controller
the entire plateau



Plateau droop

a falling heater, a slow leak
the late plateau



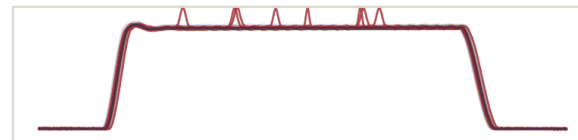
Excess overshoot

control-loop tuning gone soft
the settling window



Glitch spike

an arc, an ESD event, a comms dropout
a handful of isolated ticks



Noise burst

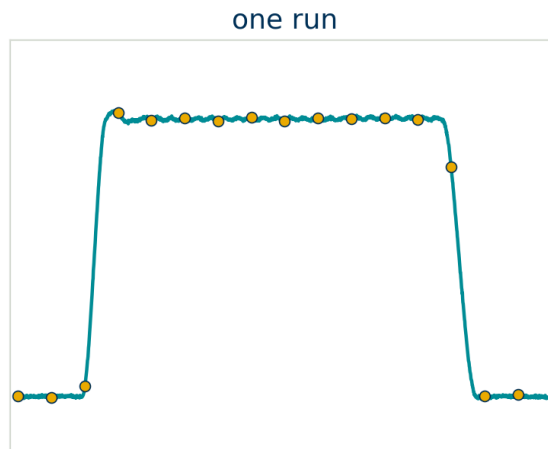
control instability, a loose connector
a window of the plateau



These traces are shown so the failure modes are concrete. No measurement in this lecture is computed from them.

A run is a point in 2000-dimensional space

Reading the trace off the time axis and stacking the samples turns each run into one long vector.



```
[ 21.1, 21.2, 20.3, 21.0, 21.5, 20.5, ...  
..., 20.3, 20.0, 19.4, 19.9 ]
```

2000 numbers.

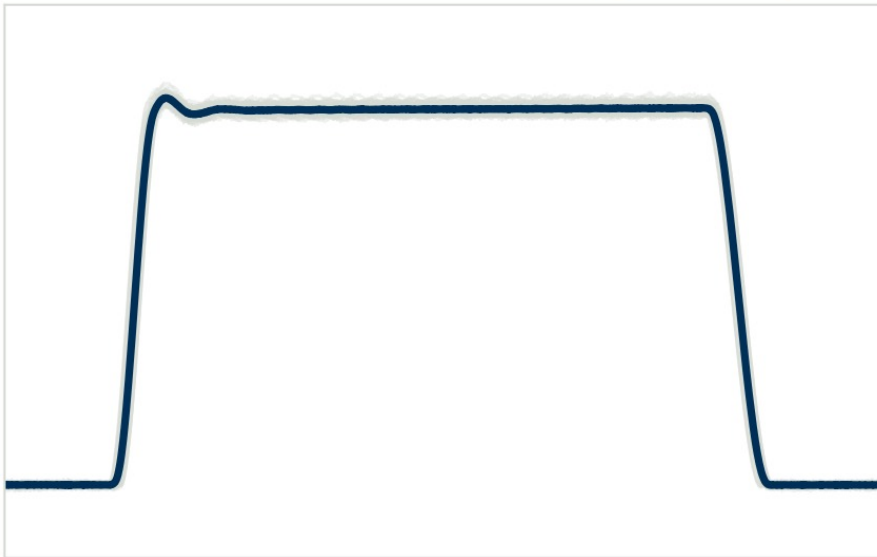
One run is one vector of length 2000.
A 128x128 image is a vector of length 16,384.

One hundred runs give a 100×2000 matrix. The rows are runs and the columns are time points. Every method in the next two lectures operates on this matrix.

Step 1 — average the runs

The mean of the hundred vectors is itself a trace, and it carries everything the runs have in common.

the mean run — add all 100 up, divide by 100



The mean run

Computed column by column. Sample k of the mean is the average of sample k across the hundred runs.

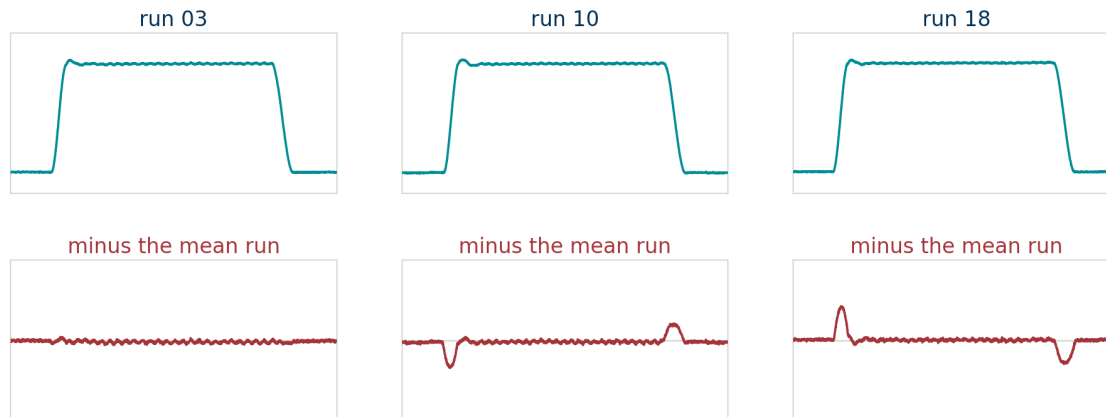
It sits close to the golden trace. The distance between them is 46 units, against 165 for a typical single run.

Subtracting it leaves only what differs between runs.

Step 2 — subtract the mean

What remains for each run is its deviation from the average run.

Every run = the mean run + a small leftover. PCA only ever looks at the leftovers.



Reading the deviations

Large at the two ramps. A few ticks of timing shift moves the signal a long way where it is changing fast.

Small on the plateau and at idle. The signal is flat there, so a timing shift changes very little.

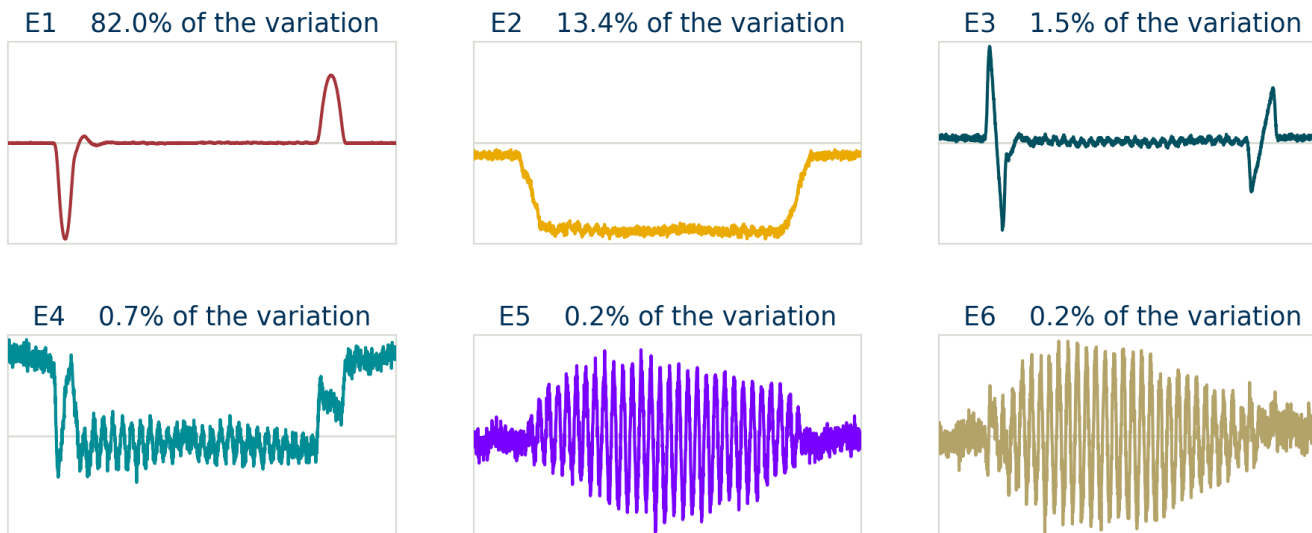
One hundred deviation curves, one per run.

The deviations are what PCA is given. The mean run is stored once and added back at the end.

Step 3 — find the patterns the deviations share

PCA returns an ordered set of trace-shaped patterns. Each one is a direction along which the runs differ.

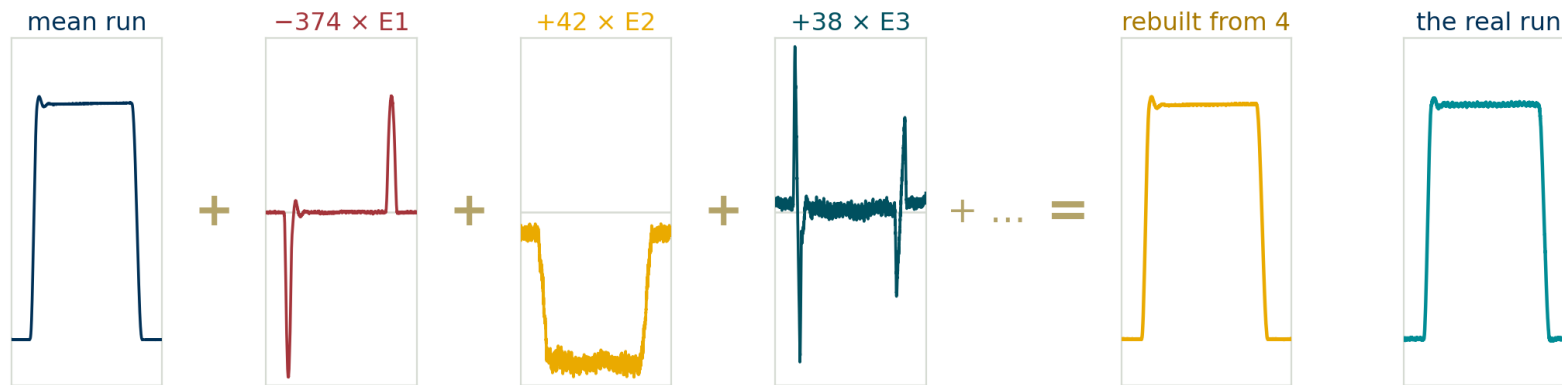
The eigentraces — the same object as an eigenface, drawn as a signal



These are called eigentraces, by analogy with the eigenfaces used for face images. The construction is identical.

Step 4 — rebuild any run from a few numbers

One run equals the mean run plus a measured amount of each eigentrace.



the whole method, in five lines

```
mu = X.mean(0)
U, s, Vt = np.linalg.svd(X - mu, full_matrices=False)
P = Vt[:k].T          # k eigentraces
coef = (X - mu) @ P    # k numbers per run
X_hat = mu + coef @ P.T # rebuilt runs
```

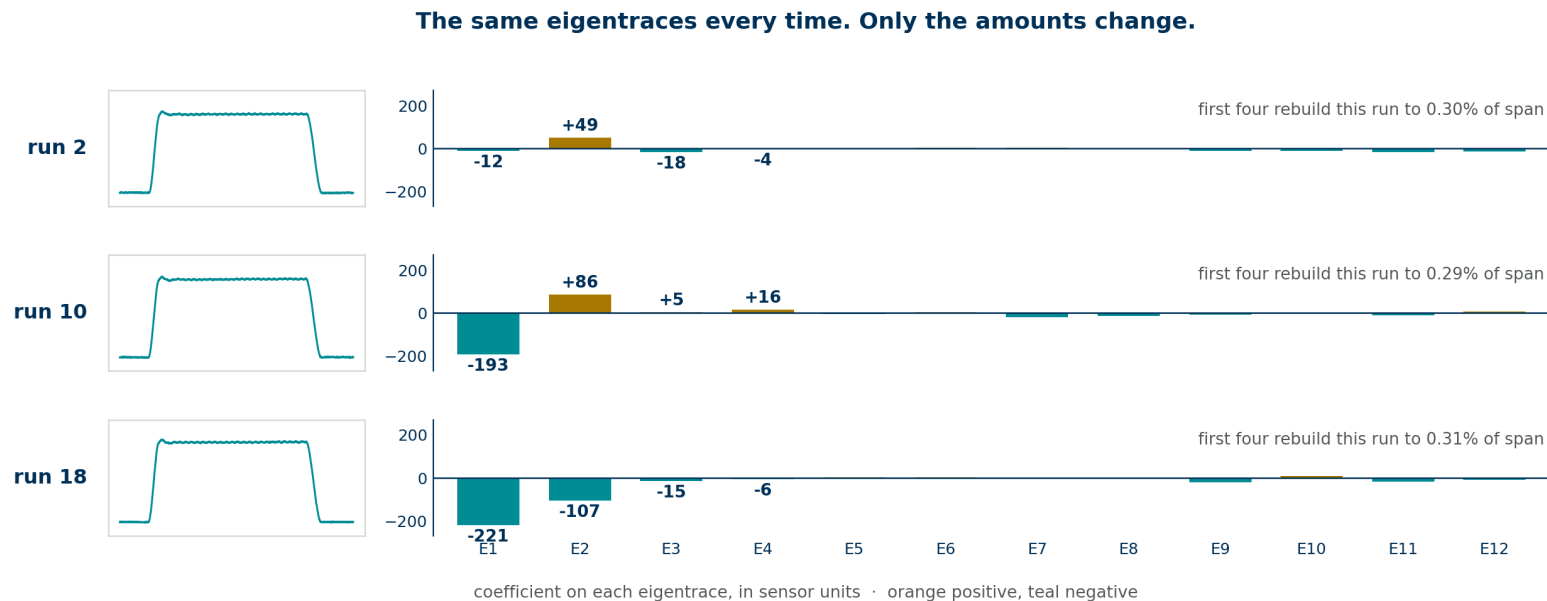
Coefficient spread across the hundred runs

E1 ± 218 E2 ± 88 E3 ± 30 E4 ± 20 E5 ± 12 E6 ± 10 units

Variance share: 82.0%, 13.4%, 1.5%. The first three carry 96.9%.

Every run is the same eigentraces in different amounts

Three runs from the hundred. The set of eigentraces is fixed. What distinguishes one run from another is the list of coefficients.



Run 2 starts close to the average, so its E1 coefficient is small. Run 18 starts late and sits low, giving a large negative E1 and a large negative E2. Beyond E4 the coefficients are small enough that dropping them changes the rebuilt run by about 0.3% of its span.

What each eigentrace changes

Add a multiple of one eigentrace to the mean run and watch which part of the trace moves.

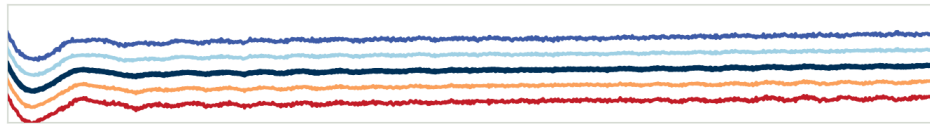
E1

Turn E1: the whole run slides earlier and later
the shape is untouched · zoomed on the rising edge



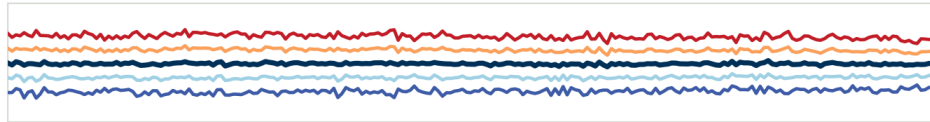
E2

Turn E2: the plateau rises and falls
the timing is untouched · zoomed on the plateau



E4

Turn E4: the whole floor lifts and drops
nothing else moves · zoomed on the idle baseline



Reading the panels

E1 slides the whole trace left and right along the time axis.

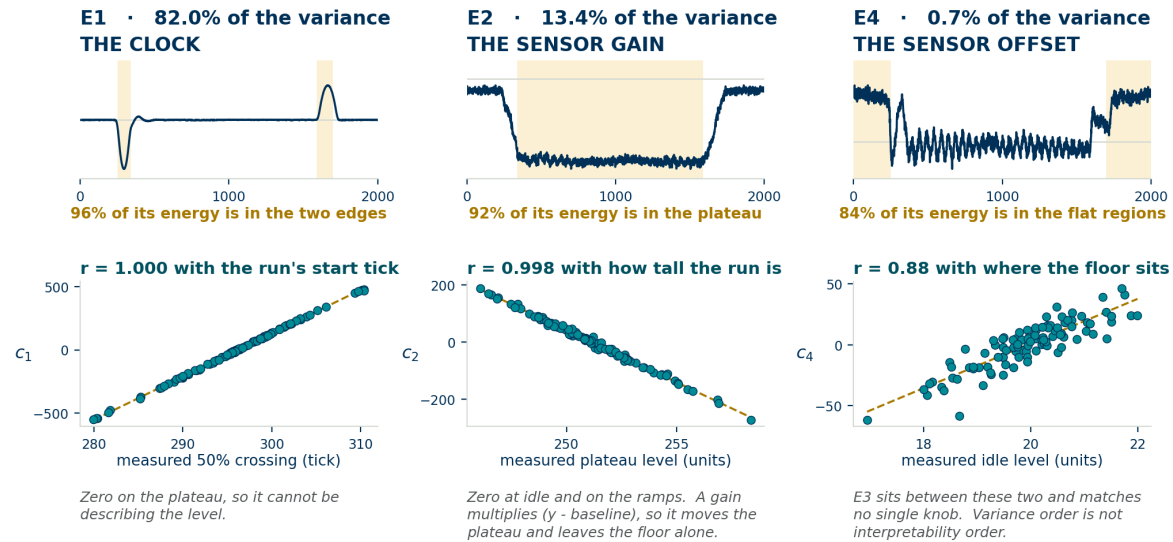
E2 raises and lowers the plateau.

E4 moves the idle level at the two ends.

E3, E5 and E6 produce small changes with no single obvious target.

What each eigentrace measures

Two independent checks: where the pattern puts its energy, and what its coefficient correlates with.



Where the energy sits

E1: 96% in the two ramps, 1% on the plateau

E2: 92% on the plateau E4: 84% in the flat regions

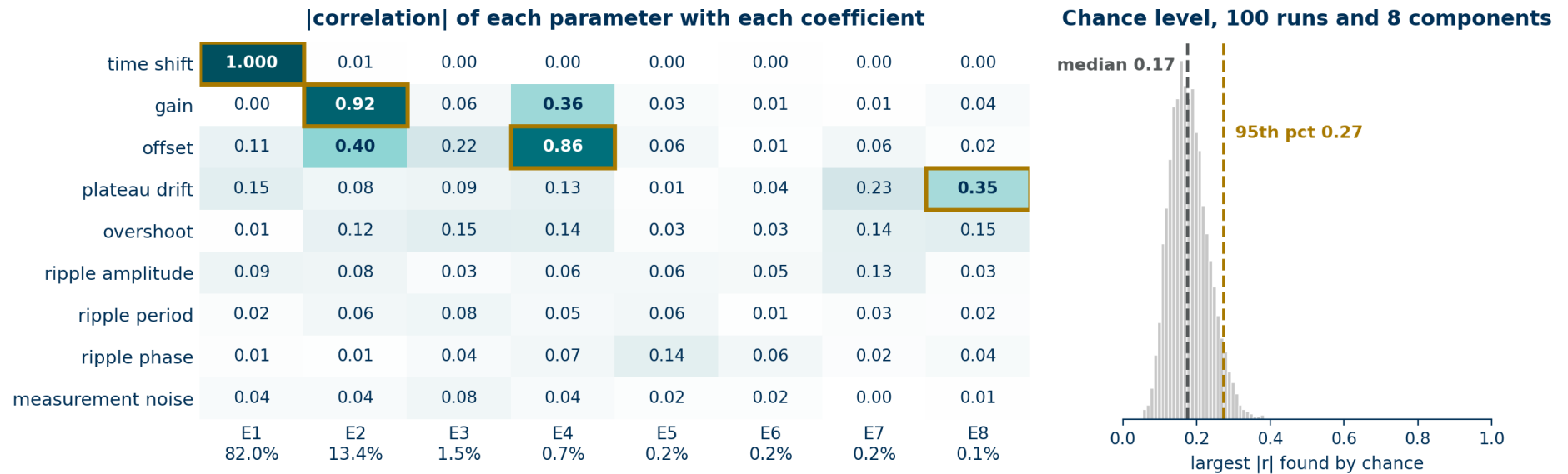
What the coefficient tracks

E1 with measured start tick: $r = 1.000$

E2 with plateau level: $r = 0.998$ E4 with idle level: $r = 0.88$

Which run-to-run parameters PCA recovered

Every parameter the generator draws per run, correlated against every coefficient across the hundred runs.



Recovered · boxed cells

Time shift E1, gain E2, offset E4, plateau drift E8. Each of these acts as one fixed shape multiplied by a number, which is what a component is.

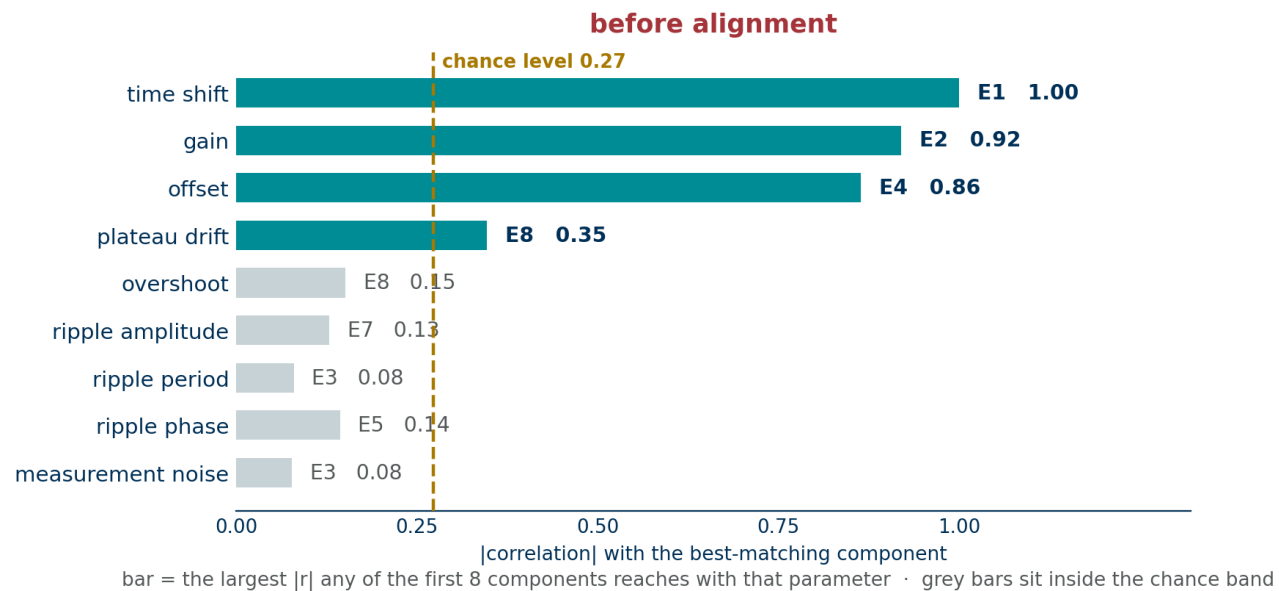
Not recovered

Overshoot, ripple amplitude, ripple period, ripple phase and measurement noise. Every correlation they reach is below the chance level of 0.27.

E3 carries 1.5% of the variance and matches nothing. Variance order and interpretability order are separate.

Which parameters the model actually captured

The same nine parameters as the previous slide, reduced to the single strongest correlation each one reaches across the first eight components.



Four parameters cleared the chance level

Time shift E1 at 1.00, gain E2 at 0.92, offset E4 at 0.86 and plateau drift E8 at 0.35.

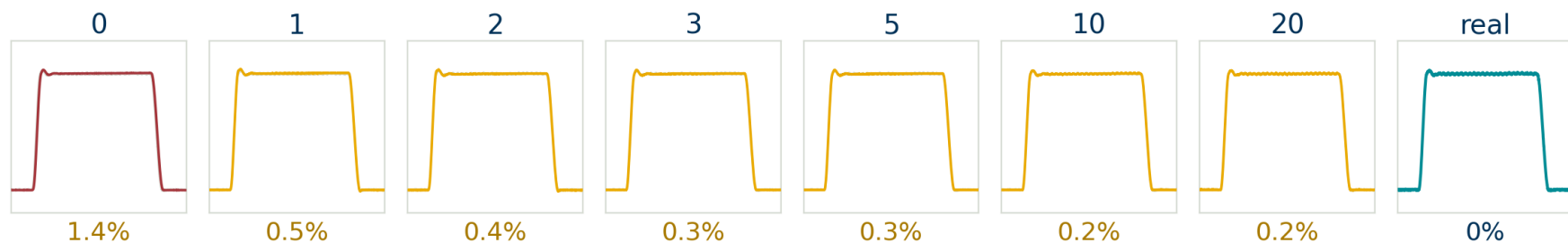
Five did not

Overshoot, ripple amplitude, ripple period, ripple phase and measurement noise all stay below 0.16.

The first component carries 82% of the variance and reports the one parameter with no bearing on wafer quality.

How many numbers a run needs

Rebuild one run from k eigentraces and measure the error against its span. The last two panels use shapes the model never saw.



above each panel: how many eigentraces were used · below: average error, % of the signal span

0.3%

three components, one training run

0.20%

a held-out healthy run

7.0%

a square pulse

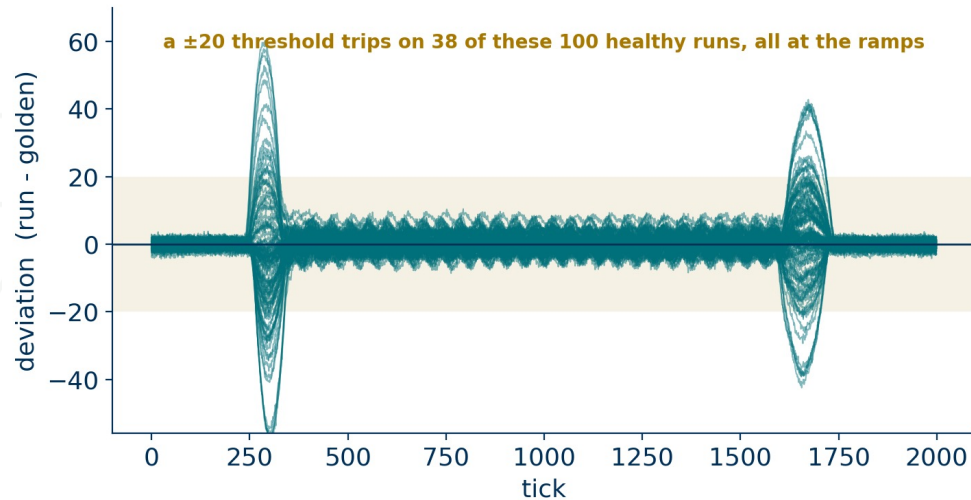
22.6%

pure noise

The model holds ingredients for healthy runs and for nothing else. The leftover error is the squared prediction error, written **SPE**. Lecture 3 turns it into half of the detector.

Detector 1 — distance from the reference trace

Subtract the reference shape from each run and threshold the largest gap.



method

```
resid = run - golden  
flag  = np.abs(resid).max() > 20
```

Result on the hundred healthy runs

38 of 100 runs exceed the threshold. Every one of them passed the health gate.

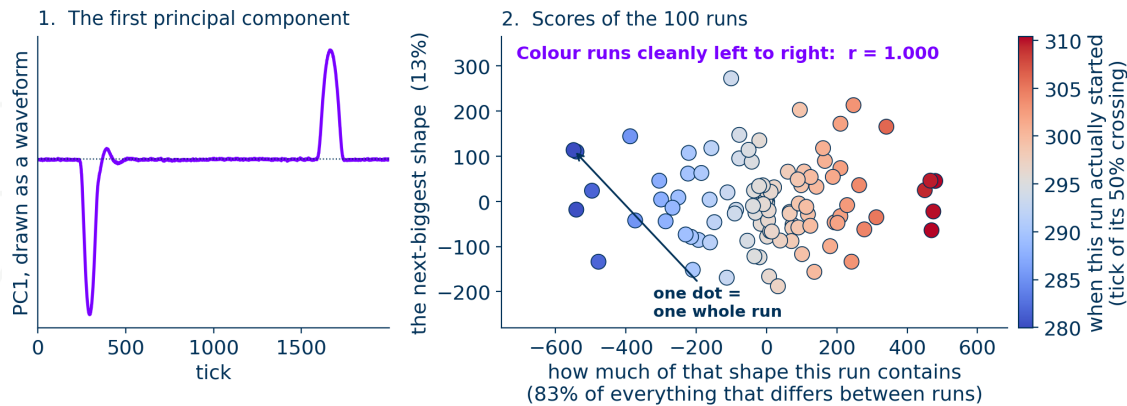
Peak residual averages 20 units.

The residual reaches ± 60 units at the ramps and ± 10 on the plateau.

The threshold that clears the ramps is 20 units, which is wider than the 10-unit setpoint fault from slide 7.

Detector 2 — PCA scores

Drop the hand-built reference. Score the runs on their first two principal components and look for outliers.



method

```
Z = PCA(n_components=2).fit_transform(X)
```

Result on the same hundred runs

PC1 carries 82.0% of the variance.

PC1 orders the runs by start time, $r = 0.9997$.

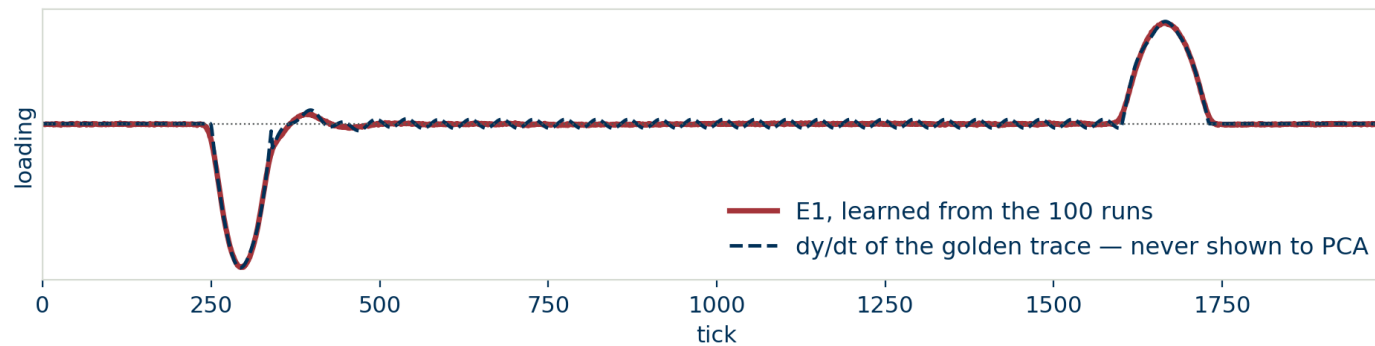
The spread along PC1 is ± 218 units.

Runs at the ends of PC1 are the earliest and latest starters, and all one hundred are healthy.

Slides 14 to 24 asked PCA to describe the hundred runs. This slide asks it to rank them. The computation is the same and the question is different.

Both results come from the start time

PCA never saw the golden trace. From one hundred runs it recovered the derivative of the golden trace, with cosine similarity 0.995.



Why the residual has two lobes

Plateau $dy/dt \approx 0$, so a few ticks of shift cost almost nothing.

Ramp dy/dt reaches 3.9 units per tick, so 6 ticks of shift produce about 23 units.

The lobes sit exactly on the two ramps.

Why PC1 has this shape

A shifted copy of a trace is, to first order,

$$y(t + \delta) \approx y(t) + \delta \cdot y'(t)$$

One hundred randomly shifted runs therefore spread along $y'(t)$, and spread is larger than everything else in the data.

Three ways to handle the start-time variation

The start time is measurable, varies by ± 15 ticks, and has no bearing on wafer quality.

Raise the threshold

Loosen it until the ramps stop tripping. The 20-unit threshold becomes 45 or more, and a 10-unit plateau offset passes through it.

Detection power is lost across the whole plateau.

Score the plateau only

Mask the ramps and the settling window and threshold what is left. The ramps stop contributing residual.

Slow ramp and excessive overshoot become undetectable.

Align each run first

Estimate the shift for each run, undo it, then score. One number per run, no training and no tuning.

Both detectors are repaired by the same step.

Estimating the shift and removing it keeps the ramps available for detection and leaves the plateau threshold tight.

Four ways to align a batch trajectory

All four estimate a time correction per run and apply it before any model is fitted.

Landmark alignment

Detect a repeatable event in each run, such as the tick where the signal first crosses half of its span, and shift the run so that event lands at a fixed index.

One parameter per run. Used for the rest of this lecture.

Cross-correlation

Slide each run against the reference and take the lag with the highest correlation.

One parameter per run. Handles pure translation.

Dynamic time warping

Find a monotone mapping between the run's time axis and the reference's.

Many parameters. Corrects stretching as well as shifting, and can absorb a real fault.

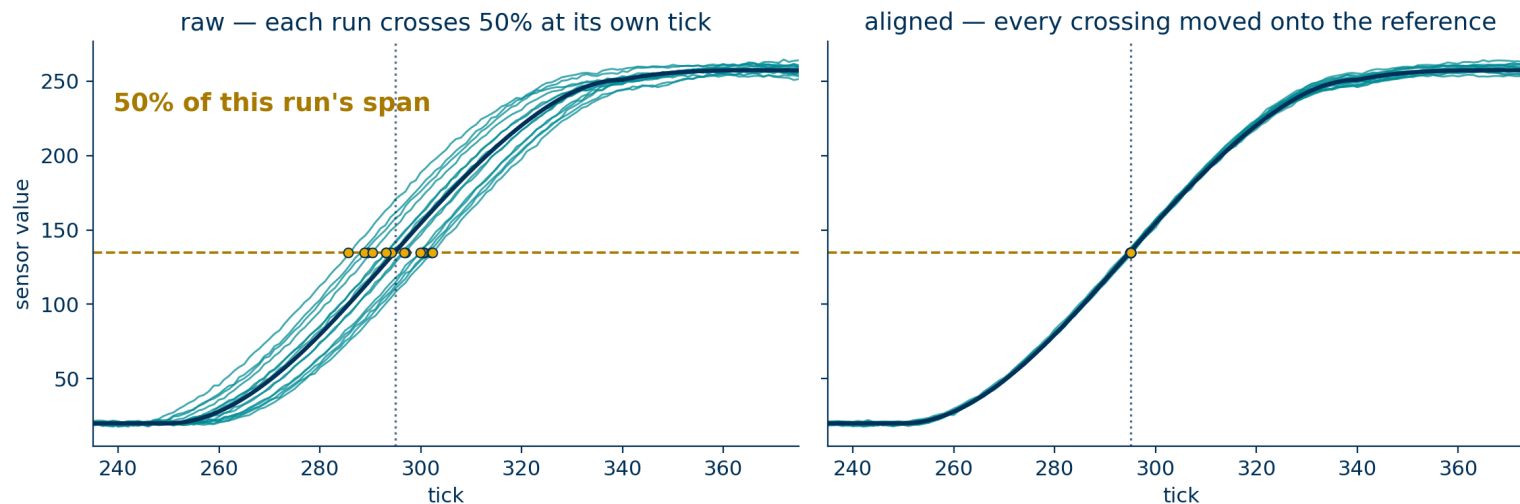
Resample to a fixed length

Normalise each phase to a common duration using the recipe step boundaries.

Needs the step boundaries from context data.

Landmark alignment, step by step

The landmark is the first tick at which the run crosses half of its span on the way up.



one run

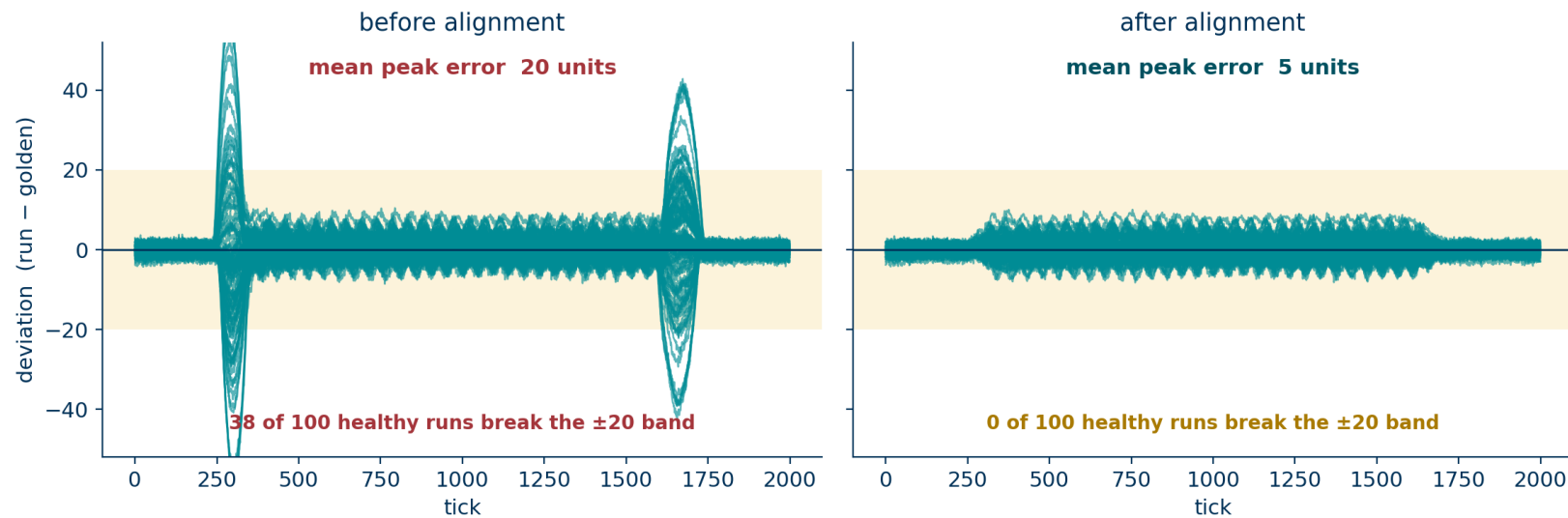
```
half = (run.max() + run.min()) / 2
k = np.argmax(run > half) # landmark tick
out = np.roll(run, REF_TICK - k) # shift onto the reference
```

Cost of the step

One integer per run, computed from the run itself.
No training set, no fitted parameters, no threshold.

Detector 1 after alignment

The same residual rule, applied to aligned runs.



38 $\rightarrow$ 0

healthy runs exceeding ± 20 , out of 100

20 $\rightarrow$ 5

mean peak residual, units

$\pm 60 \rightarrow \pm 10$

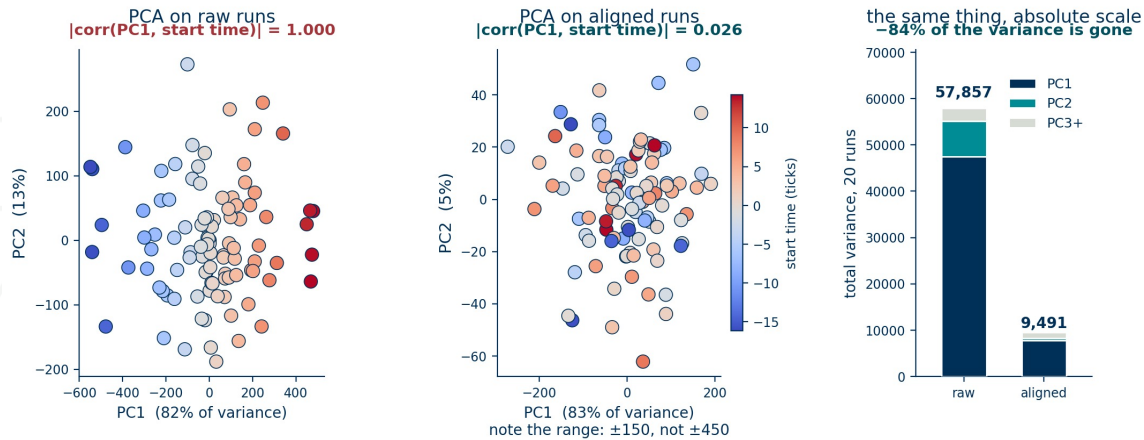
largest residual anywhere

20 $\rightarrow$ 8

threshold the plateau can now support

Detector 2 after alignment

The same PCA, applied to aligned runs.



Measured change

Total variance 57,900 $\rightarrow$ 9,500, a drop of 84%

corr(PC1, start time) 0.9997 $\rightarrow$ 0.026

PC1 spread $\pm 218 \rightarrow \pm 88$ units

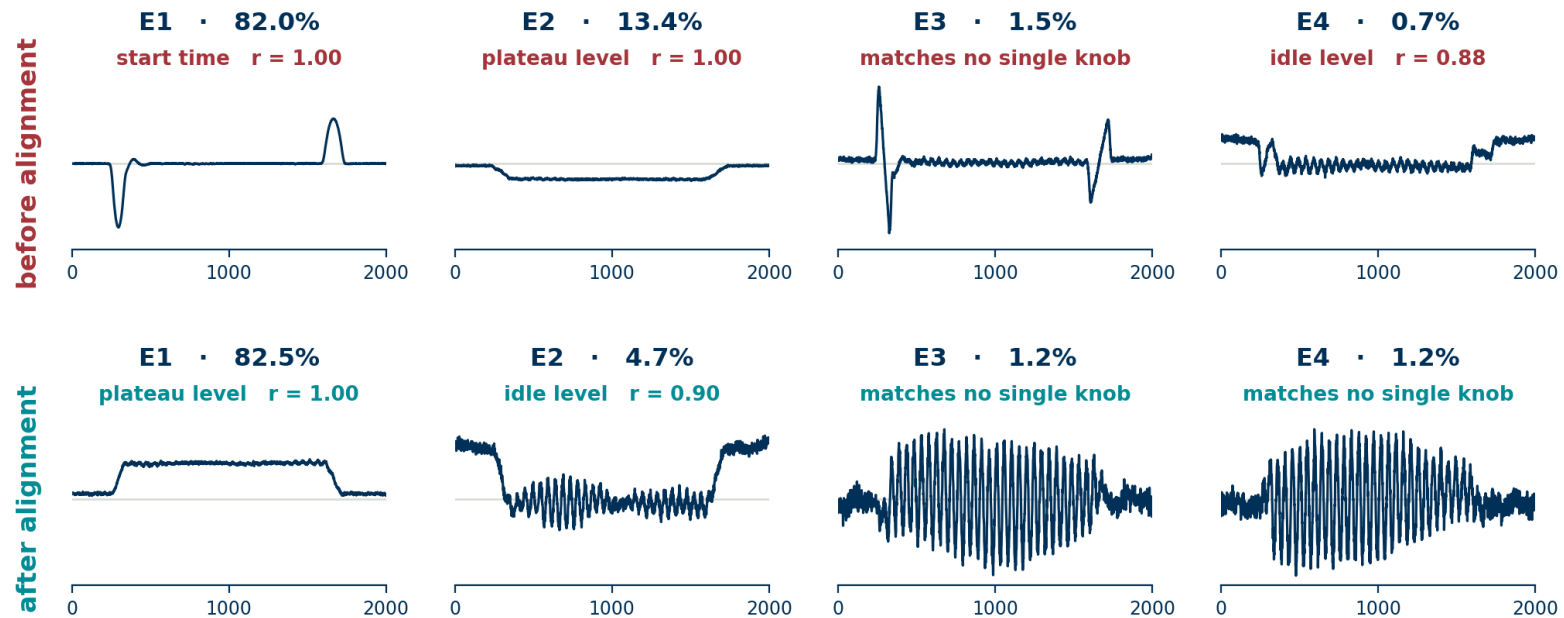
The new PC1 matches the old PC2 at cosine 0.998, with variance 7,750 $\rightarrow$ 7,830

PC1 variance share 82.0% $\rightarrow$ 82.5%

The share is unchanged because the numerator and the denominator fall together. The absolute variance is the quantity that reports what alignment did.

What the eigentraces became

The same PCA, run on the raw runs and then on the aligned runs. Percentages are the share of variance each component carries.



Before

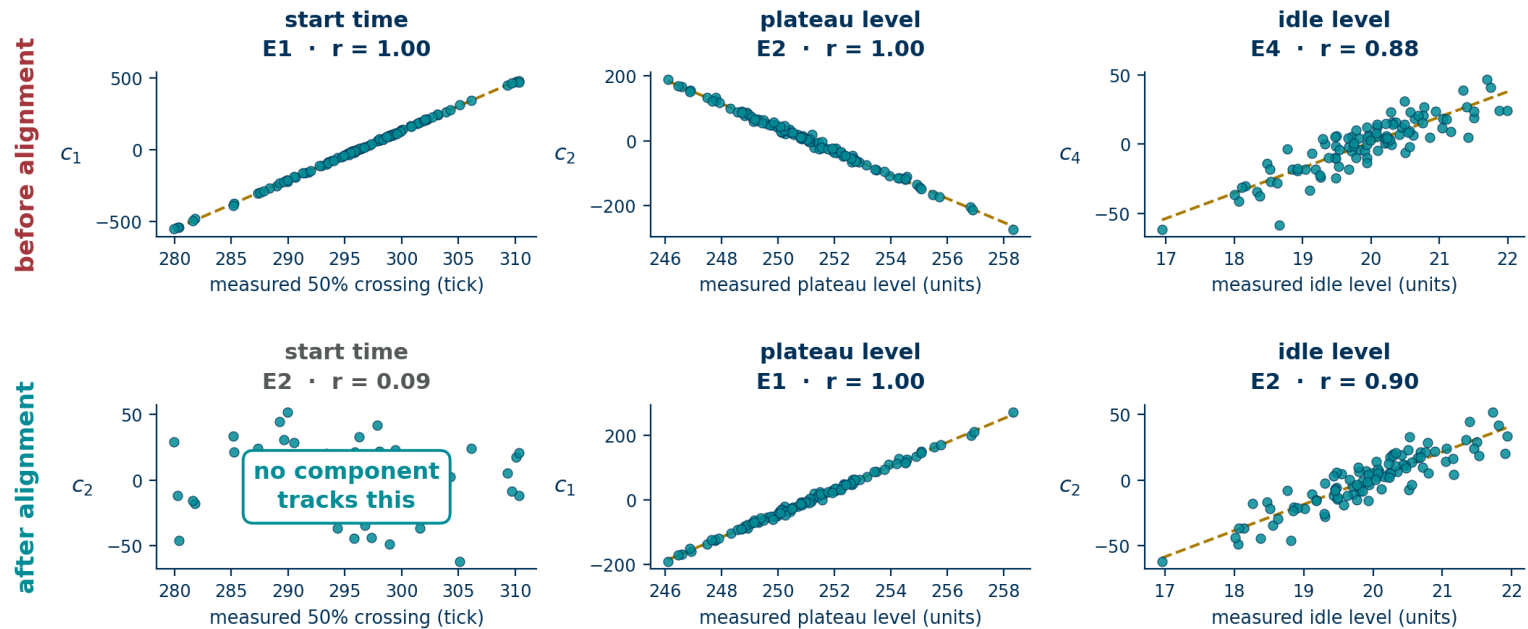
E1 and E3 are spikes at the two ramps. Both describe timing.

After

Nothing edge-localised remains. E1 is the plateau level, E2 the idle level.

What the components track, before and after

The same three quantities measured off the raw traces, plotted against the component that follows each one most closely.

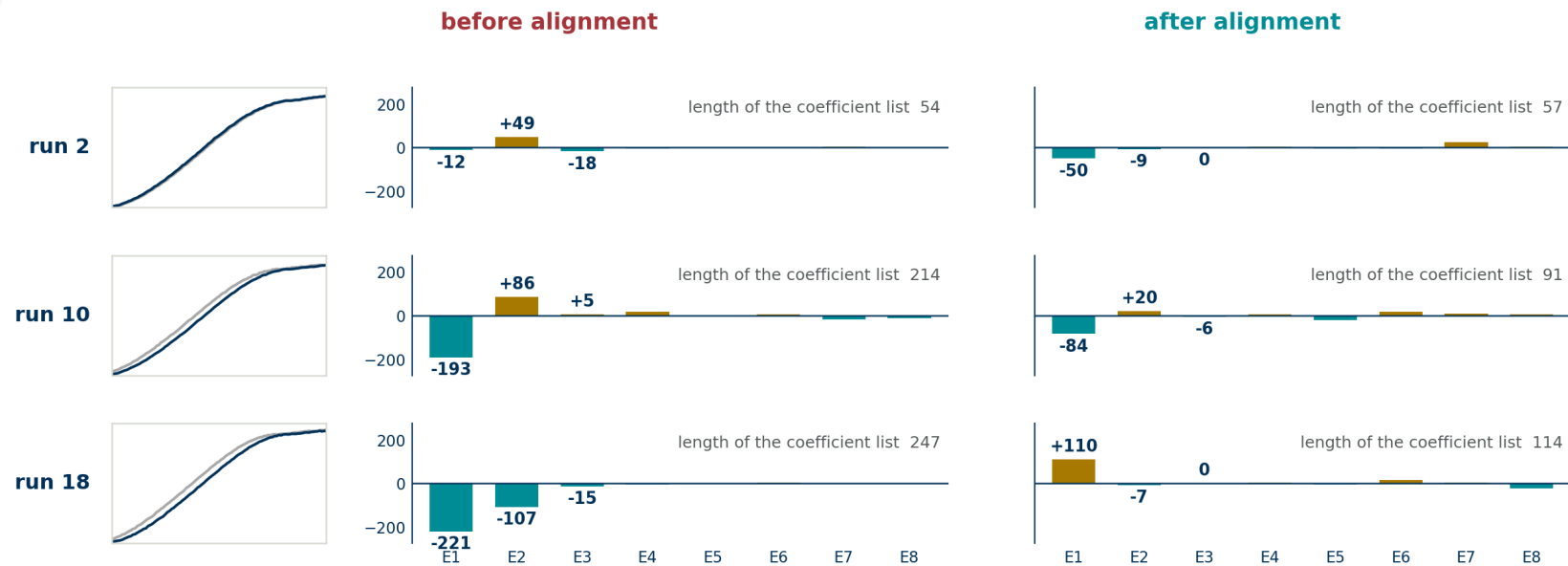


Before: E1 is the clock, and the plateau level has to wait for E2.

After: no component reaches $r = 0.1$ with the start time, and the plateau level leads.

The same runs, decomposed before and after

Runs 2, 10 and 18 again. The left column zooms the rising edge so the correction is visible; the bar charts share one vertical scale.



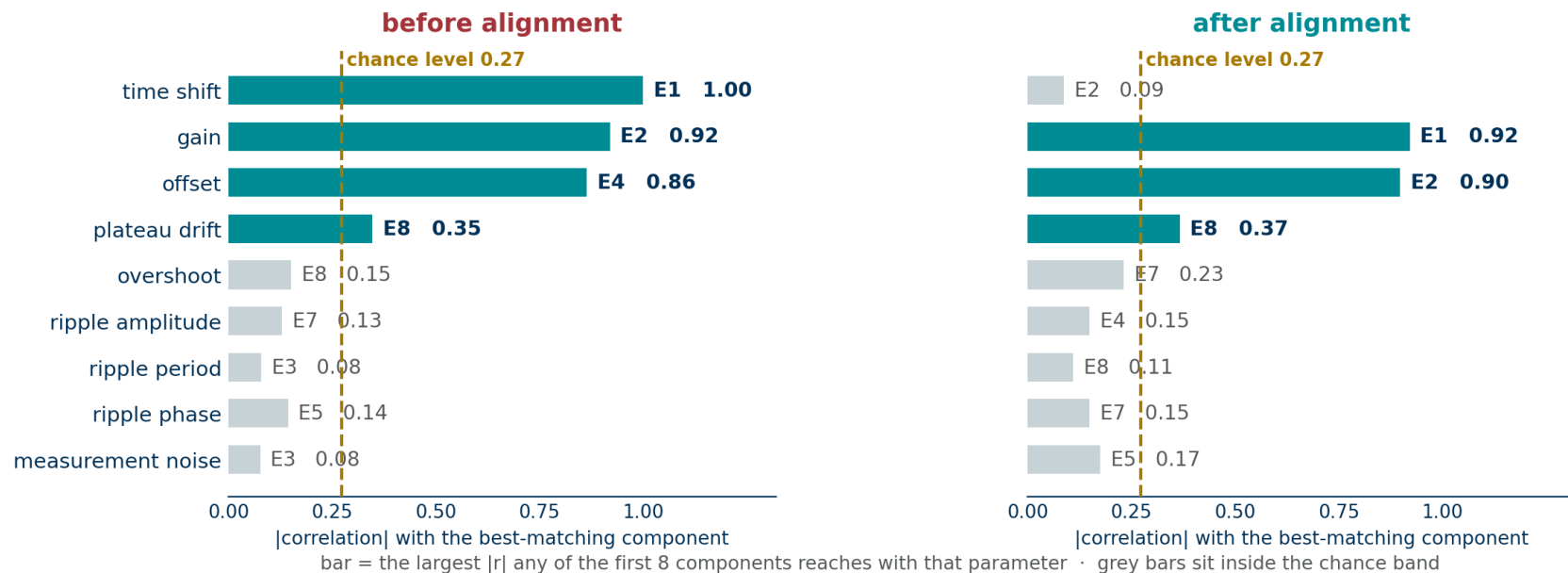
left column zooms the rising edge: grey = raw run, navy = the same run after alignment · coefficients in sensor units, shared vertical scale

Before: E1 dominates every run, and it is reporting the start time.

After: the coefficient lists shorten by more than half, and E1 now reports the plateau level.

Every generator parameter, before and after alignment

All eight run-to-run knobs plus measurement noise, each searched against the first eight components. The bar is the largest correlation any component reaches.



Recovered as components

Before: time shift E1, gain E2, offset E4, drift E8. **After:** gain E1, offset E2, drift E8.

Never recovered

Overshoot, ripple amplitude, ripple period, ripple phase and measurement noise stay inside the chance band at every step.

Alignment removes the time shift from the model and promotes everything below it. No parameter that was invisible becomes visible, and no parameter that was visible is lost.

Summary

Four results, measured on one hundred healthy runs and no labelled faults.

- 1 A fault can hold a correct shape at the wrong level.**
The setpoint offset moved the plateau by 10 units on a 230-unit span, which is about four times the healthy plateau spread, and stayed within normal variation everywhere else.
- 2 Normal is defined from good runs, because those are the runs you have.**
Faults are rare, discovered late, and open-ended, so the model of normal is built from healthy data alone.
- 3 Two thousand samples per run reduce to about four numbers.**
The mean run plus three eigentraces reproduce a healthy run to 0.3% of its span, and the leftover error separates real runs from shapes the model has no ingredients for.
- 4 Both detectors measured the start time.**
The residual lobes and the first principal component both follow the derivative of the golden trace. Aligning each run first took peak residual from 20 to 5 units and $\text{corr}(\text{PC1}, \text{start time})$ from 0.9997 to 0.026.

Next lecture: the detector a fab runs in production, the control limits that go with it, and how it is scored.